Supervised assets

A proposal for issuer-freezable assets on Sequentia, shaped by what the analysis ruled out. 2026-08-12, for Andreas and Alberto.

Proposal: add a declared asset class, *supervised*, whose issuer can freeze holders by consensus rule. Freezability is committed into the asset id at issuance, so it is permanent, visible before anyone acquires the asset, and impossible to add later. An asset not born supervised can never become supervised, and nothing about ordinary assets changes.

The design's one unusual move is what makes it safe: **a supervised asset may only live in single-owner outputs, and may never be blinded.** Both restrictions are enforced by consensus at output creation. That is what removes every way a freeze could strand a third party or destroy a timelocked remedy, which is what sank the obvious version of this feature.

The trade is explicit and belongs to the issuer, not to us: a supervised asset cannot enter Lightning channels, the covenant order book, or HTLC swaps. It is an ordinary transferable asset in every other respect, including paying its own fees. **Circle would choose, per asset, between composable-and-unfreezable and supervised-and-freezable.** For USDC that choice is still open, because the asset has zero supply and no deposits.

1. What this is for

Sequentia cannot today offer a fiat issuer the two controls its own token contract has everywhere else: a transfer-level freeze and a transfer-level pause. Co-signed custody provides them but taxes every single transfer with an online policy server, which is why it is right for securities and wrong for a dollar. Everything else on the chain, covenants included, can only bind outputs it created, so nothing reaches a holder who already holds the asset.

Only a consensus rule can reach an existing holder. The question this proposal answers is what that rule must look like to be worth having, given that a first sketch of it broke Lightning, the DEX, atomic swaps and block production.

2. The proposal

2.1 A supervised asset is declared at birth

The issuance commits an **issuer freeze key** into the asset id, alongside the contract hash that already binds the asset's name and ticker. Consequences follow from the commitment rather than from policy:

- Freezability is **immutable**. It cannot be added to an existing asset, because the id is a hash of the thing that declares it, and it cannot be removed.

- Freezability is **identifiable** before acquisition, provided the property is exposed where holders look. That is deliberately part of the feature, not a follow-up: see 4.2.
- Authority is **unambiguous**. A freeze record is valid only if signed by that key. This is what the derivation-tag idea got wrong, since the entropy it relied on is public and would have let anyone freeze anyone.

2.2 A supervised asset may only occupy single-owner outputs

Consensus rejects any transaction creating a supervised-asset output whose scriptPubKey is not a single-key form (P2WPKH, or P2TR spent by key path). No multisig, no script path, no timelocked script, no covenant.

This is the load-bearing restriction, and every one of the blockers below dissolves because of it rather than being mitigated after the fact:

What broke in the naive design	Why it cannot happen here
Freezing a Lightning funding output strands the innocent peer: no cooperative close, no force close, no HTLC resolution	A supervised asset can never be in a channel, because a funding output is a 2-of-2
A freeze outliving a timelock destroys the remedy instead of delaying it, handing a cheating peer a theft primitive	No supervised output can carry a timelock, so there is no deadline for a freeze to outlive
Freezing a resting covenant order kills the taker's fill and the maker's refund	A supervised asset cannot rest in a covenant output
A mid-flight freeze strands a swap counterparty who has already paid	A supervised asset cannot be locked in an HTLC
A blinded input hides its asset, so enforcement is either evaded or must guess	Supervised outputs are explicit by rule, so the asset is always visible to consensus

What survives is the ordinary case, which is the case that matters for a dollar: person to person, person to merchant, deposits and withdrawals at an exchange, and same-chain DEX trades, where both sides' inputs are single-owner outputs and each party can check the other's freeze status before signing.

2.3 Freezing, and unfreezing

A freeze record is an output carrying the asset id, the target scriptPubKey, and a signature by the issuer's freeze key. The set of frozen targets is derived from unspent records, exactly as the staking and delegation registries already derive consensus-relevant state from the UTXO set, so a reorg inverts it with the same apply-and-revert discipline rather than bespoke rollback code. The issuer unfreezes by spending its own record, which means an unfreeze is as auditable as a freeze and

cannot be done quietly.

A pause is the same mechanism at asset scope: one record freezing the asset rather than a target.

2.4 Explicit-only is a feature, not only a cost

Supervised assets give up confidentiality. That is required for enforcement, since consensus cannot see which asset a blinded output holds. It is worth noting what it buys: a supervised asset's entire flow is publicly traceable, which is precisely what makes an issuer's compliance obligations tractable, and it means the supply auditor already built for the bridged standard reports exact figures for it forever.

3. What this does not do, stated plainly

Two limits an issuer must be told before adopting this, because discovering either one later would be worse than not shipping it.

It freezes coins at a script, not a person. A holder with funds spread across twenty addresses must be frozen twenty times, and the freezing transaction is public before it confirms, so an attentive target can move first. This is closer to the EVM situation than it first appears, since blacklisting an Ethereum address also leaves the holder free to use another, and enforcement there likewise depends on identifying addresses. The honest difference is aggregation: EVM holders tend to sit on one address, while a UTXO wallet rotates by default. Against a custodian, an exchange deposit address, or a wallet identified by analysis, the rule works. Against an evading self-custody holder, it is a race and sometimes the holder wins.

It is not retroactive to history. The rule takes effect at an activation height and only rejects spends after it. That is mandatory, not conservative: a rule that rejects spends must never apply to blocks produced before it existed, or a fresh sync stops dead at the first such block.

4. What must ship with it

These are not refinements. Each one is something whose absence would either break the chain or make the feature dishonest.

4.1 Mempool eviction

A spend that was valid on entry becomes invalid the moment a freeze confirms, and nothing currently removes it: the mempool drops only transactions included in a block or directly conflicting, and a freeze record conflicts with nothing. The stale transaction is then re-selected into every block template, template construction fails, and producers skip slots. The node also asserts that this cannot happen: `CTxMemPool::check` re-runs `CheckTxInputs` inside a bare `assert` commented "should always pass" (`src/txmempool.cpp:920`), so a freeze would abort nodes outright. Eviction on freeze, re-admission on unfreeze, and a reconsidered assert are part of the feature.

4.2 Discoverability

A holder must be able to answer "is this asset supervised, and is my address frozen?" before acquiring it. That means an RPC keyed by asset id, and the wallet, explorer, registry and DEX all showing it. Without this the constraint that freezability be identifiable is satisfied on paper and not in practice.

4.3 A rejection reason that tools can act on

A frozen spend must fail with a distinct, machine-readable reason, so a wallet can tell its user "the issuer has frozen this" rather than showing a generic consensus error, and so the DEX can reap orders it can no longer fill.

4.4 Activation and cutover

Height-gated, chosen above the tip at release, and rolled out with the same all-nodes-at-once discipline as the Simplicity activation and the 60-second-block fork. An un-upgraded node diverges at the gate.

5. What it means for USDC

This is the decision that makes the proposal urgent rather than theoretical. `USDC.e` exists, is registered, and has **zero supply and zero deposits**. Supervision can only be conferred at issuance, so the asset can still be re-issued as supervised today at no cost. Once real USDC is bridged, the door closes permanently.

The choice is genuinely open and it is Circle's to influence:

If USDC stays an ordinary asset	**If USDC is re-issued supervised**
Works in Lightning, the covenant order book and HTLC swaps; fully composable; optional confidentiality	Issuer can freeze and pause, matching what FiatToken does everywhere else
No freeze, ever, for anyone	No Lightning, no covenant orders, no HTLC swaps; permanently transparent
Matches Tether on Liquid, which operates with no freeze at all	Matches Circle's posture on every chain it issues natively on

My recommendation is to **build the capability and leave USDC ordinary until Circle asks otherwise**. The capability is what makes the conversation possible; committing USDC to it before Circle has expressed a requirement would pay the composability cost for a benefit nobody has yet asked for, and the bridged phase is exactly when composability earns the adoption that makes an upgrade worth their while.

6. Scope, and what I would want from Alberto

Implementation is a release of its own: issuance derivation and validation, an output-type rule, a freeze registry with apply and revert, enforcement in `CheckTxInputs`, mempool eviction, RPCs, functional tests covering reorg and eviction, and display work across the wallet, explorer, registry and DEX.

Four questions I would rather settle with Alberto before any code:

1. **Output-type whitelist.** Is P2WPKH plus P2TR key-path the right set? Allowing P2TR script-path would restore covenant and Lightning use and reintroduce every stranding problem, so I propose excluding it, but the call affects what supervised assets can ever do.
2. **Where the freeze key is committed.** Folding it into the existing contract hash is the smaller change; a distinct derivation constant makes the asset class visible from the id itself. The first is simpler, the second is more honestly "identifiable".
3. **Whether a pause needs its own record type** or is simply a freeze naming the asset rather than a script.
4. **Whether supervised assets may pay fees.** They can technically, but a frozen holder's fee output interacts with the producer's fee handling and deserves a deliberate answer.

7. Provenance

This design is what remained after six independent analyses were run against the real trees and their blocker-rated findings were attacked by separate reviewers. The earlier sketch, freezing arbitrary scripts of any type, failed on five counts: whole-transaction rejection stranding innocent counterparties, timelock destruction, confidential evasion, mempool wedging, and an authorisation hole. Each is addressed above by construction rather than by mitigation, which is the difference between this proposal and that one.

Two claims were re-verified by hand rather than taken on an analyst's word: the mempool assert, and that `CheckTxInputs` rejects whole transactions rather than individual inputs. One reported finding was discarded as an artifact of a stale checkout: `~/Sequentia` sits on `pos-exprace` and still shows Simplicity inactive, while master has it active and testnet proved it in block 89860. That checkout should be brought up to date, since other sessions read it as ground truth.

Prepared by Saba. No consensus code has been written; a sketch produced during analysis was reverted unbuilt.